Ejercicio 1: Factorial

Prompt redactado:

Quiero implementar en Java una función recursiva que calcule el factorial de un número entero positivo n ($n!$).

Este problema puede resolverse recursivamente porque el factorial de n se define en términos del factorial del número anterior: $n! = n \times (n-1)!$.

El caso base es cuando n vale 0 o 1, porque por definición matemática $0! = 1$ y $1! = 1$, deteniendo así la recursión.

El caso recursivo es $n \times \text{factorial}(n-1)$, porque en cada llamada el problema se reduce restando 1 a n hasta aproximarse a 1 o 0.

La función debe devolver un número entero con el resultado del factorial.

Quiero que el código tenga comentarios explicando el caso base, el caso recursivo y el funcionamiento de la pila de llamadas.

También quiero probarlo con estos casos de prueba:

- Caso 1: $n = 0$ (debe devolver 1)
- Caso 2: $n = 1$ (debe devolver 1)
- Caso 3: $n = 5$ (debe devolver 120)

Ejercicio 2: Suma de los primeros N números

Prompt redactado:

Quiero implementar en Java una función recursiva que calcule la suma de los números enteros desde n hasta 1.

Este problema puede resolverse recursivamente porque la suma de los primeros n números se puede pensar como $n + \text{suma}(n-1)$.

El caso base es cuando $n = 0$ (o $n = 1$), porque la suma de 0 elementos es 0 y constituye el escenario más simple sin más sumas que realizar.

El caso recursivo es $n + \text{suma}(n-1)$, porque en cada llamada se suma el valor actual n y el problema se reduce decrementando n en 1.

La función debe devolver el total acumulado como un entero.

Quiero que el código tenga comentarios explicando el caso base, el caso recursivo y la validación de su correcto funcionamiento.

También quiero probarlo con estos casos:

- Caso 1: $n = 0$ (debe devolver 0)

- Caso 2: $n = 1$ (debe devolver 1)
- Caso 3: $n = 5$ (debe devolver 15, ya que $5 + 4 + 3 + 2 + 1 = 15$)

Ejercicio 3: Multiplicación mediante sumas

Prompt redactado:

Quiero implementar en Java una función recursiva que realice la multiplicación de dos números enteros a y b mediante sumas repetidas, sin utilizar el operador $*$.

Este problema puede resolverse recursivamente porque multiplicar $a \times b$ equivale a sumar a un número b de veces: $a + (a \times (b - 1))$.

El caso base es cuando el factor $b = 0$, porque cualquier número multiplicado por 0 da 0.

El caso recursivo es $a +$ multiplicar$(a, b - 1)$, porque reduce progresivamente el segundo parámetro b restándole 1 en cada llamada.

La función debe devolver el resultado entero del producto.

Quiero que el código tenga comentarios explicando el caso base, el caso recursivo y la reducción del parámetro.

También quiero probarlo con estos casos:

- Caso 1: $a = 5, b = 0$ (debe devolver 0)
- Caso 2: $a = 7, b = 1$ (debe devolver 7)
- Caso 3: $a = 4, b = 3$ (debe devolver 12)

Ejercicio 4: Potencia

Prompt redactado:

Quiero implementar en Java una función recursiva que calcule la potencia de una base a elevada a un exponente b (a^b) sin utilizar Math.pow.

Este problema puede resolverse recursivamente porque la potencia puede expresarse como multiplicaciones sucesivas: $a^b = a \times a^{(b-1)}$.

El caso base es cuando el exponente $b = 0$, porque todo número elevado a la potencia 0 da como resultado 1.

El caso recursivo es $a \times$ potencia$(a, b - 1)$, porque en cada paso el exponente b se reduce en 1 hasta alcanzar 0.

La función debe devolver el valor entero resultado de la potencia.

Quiero que el código tenga comentarios explicando el caso base, el caso recursivo y el retorno en cada llamado.

También quiero probarlo con estos casos:

- Caso 1: base = 5, exponente = 0 (debe devolver 1)
- Caso 2: base = 3, exponente = 1 (debe devolver 3)
- Caso 3: base = 2, exponente = 4 (debe devolver 16)

Ejercicio 5: Conteo regresivo

Prompt redactado:

Quiero implementar en Java una función recursiva de tipo void que imprima por consola un conteo regresivo desde n hasta 0.

Este problema puede resolverse recursivamente porque en cada llamada se muestra el número actual n y luego la función se llama a sí misma para mostrar los números restantes $(n - 1)$.

El caso base es cuando $n < 0$, momento en el cual la función simplemente finaliza con return.

El caso recursivo consiste en imprimir n y luego invocar a conteoRegresivo$(n - 1)$, acercándose al caso base al restar 1 (se advierte que usar $n + 1$ generaría una recursión infinita y un error de StackOverflow).

La función no devuelve ningún valor (void), únicamente ejecuta impresiones por consola.

Quiero que el código tenga comentarios explicando el caso base, la acción del caso recursivo y la diferencia de no retornar valor.

También quiero probarlo con estos casos:

- Caso 1: $n = 0$ (imprime: 0)
- Caso 2: $n = 3$ (imprime: 3, 2, 1, 0)
- Caso 3: $n = 5$ (imprime: 5, 4, 3, 2, 1, 0)

Ejercicio 6: Contar dígitos

Prompt redactado:

Quiero implementar en Java una función recursiva que cuente la cantidad de dígitos de un número entero positivo n.

Este problema puede resolverse recursivamente porque al dividir de manera entera un número por 10 se elimina su último dígito, reduciendo el problema.

El caso base es cuando el número es menor que 10 $(n < 10)$, porque todo número de un solo dígito tiene una longitud de 1.

El caso recursivo es $1 +$ contarDigitos$(n/10)$, porque cada división entera remueve un dígito sumando 1 a la cuenta total.

La función debe devolver la cantidad total de dígitos como un entero.

Quiero que el código tenga comentarios explicando el caso base, la división entera y la acumulación de la llamada recursiva.

También quiero probarlo con estos casos:

- Caso 1: $n = 7$ (debe devolver 1)
- Caso 2: $n = 45$ (debe devolver 2)
- Caso 3: $n = 1289$ (debe devolver 4)

Ejercicio 7: Sumar dígitos

Prompt redactado:

Quiero implementar en Java una función recursiva que calcule la suma de todos los dígitos de un número entero positivo n.

Este problema puede resolverse recursivamente aislando el último dígito con la operación módulo $(n\%10)$ y sumándolo a la suma de los dígitos restantes obtenida con división entera $(n/10)$.

El caso base es cuando $n < 10$, porque la suma de los dígitos de un número de una sola cifra es el mismo número n.

El caso recursivo es $(n\%10) +$ sumarDigitos$(n/10)$, combinando el último dígito con el resultado de la función aplicada al resto del número.

La función debe devolver el resultado de la suma total de los dígitos.

Quiero que el código tenga comentarios explicando el uso de módulo, división entera y la combinación de resultados en la pila.

También quiero probarlo con estos casos:

- Caso 1: $n = 5$ (debe devolver 5)
- Caso 2: $n = 24$ (debe devolver 6)
- Caso 3: $n = 1234$ (debe devolver 10, ya que $1 + 2 + 3 + 4 = 10$)

Ejercicio 8: Invertir una palabra

Prompt redactado:

Quiero implementar en Java una función recursiva que invierta una cadena de texto (String).

Este problema puede resolverse recursivamente concatenando la inversión de la subcadena que excluye el primer carácter con dicho primer carácter colocado al final.

El caso base es cuando la cadena es vacía o tiene un solo carácter (palabra.length() <= 1), ya que una cadena de esa longitud ya está invertida.

El caso recursivo es invertir$(\text{palabra.substring}(1)) + \text{palabra.charAt}(0)$, separando el primer carácter y reduciendo la longitud en cada llamada.

La función debe devolver un nuevo String con la palabra invertida.

Quiero que el código tenga comentarios explicando el caso base, el despiece de la cadena con substring y la reconstrucción al regresar de la recursión.

También quiero probarlo con estos casos:

- Caso 1: "a" (debe devolver "a")
- Caso 2: "java" (debe devolver "avaj")
- Caso 3: "recursion" (debe devolver "noisrucer")

Ejercicio 9: Palíndromo

Prompt redactado:

Quiero implementar en Java una función recursiva que determine si una palabra es un palíndromo (se lee igual de izquierda a derecha y viceversa).

Este problema puede resolverse recursivamente comparando si el primer y el último carácter son iguales; si lo son, se verifica de forma recursiva la subcadena intermedia sin los extremos.

El caso base es si la cadena tiene longitud 0 o 1 (devuelve true), o bien si el primer y último carácter difieren (devuelve false de forma temprana).

El caso recursivo es esPalindromo$(\text{palabra.substring}(1, \text{palabra.length}(\ ) - 1))$, aplicando si los caracteres extremos coinciden.

La función debe devolver un valor booleano (true o false).

Quiero que el código tenga comentarios explicando los casos base, la comparación de extremos y la reducción de la cadena.

También quiero probarlo con estos casos:

- Caso 1: "reconocer" (debe devolver true)
- Caso 2: "neuquen" (debe devolver true)
- Caso 3: "algoritmos" (debe devolver false)

Ejercicio 10: Buscar un elemento en un arreglo

Prompt redactado:

Quiero implementar en Java una función recursiva que busque la posición de un número entero dentro de un arreglo sin utilizar ningún tipo de ciclo (for o while).

Este problema puede resolverse recursivamente evaluando si el elemento en la posición o índice actual coincide con el valor buscado, y si no, avanzando la búsqueda incrementando el índice en 1.

El caso base ocurre cuando se encuentra el valor (devuelve el índice actual) o cuando el índice alcanza la longitud del arreglo (devuelve -1 indicando que no se encontró).

El caso recursivo es buscar$(\text{arreglo}, \text{buscado}, \text{indice} + 1)$, reduciendo las posiciones pendientes de inspeccionar.

La función debe devolver la posición entera del elemento dentro del arreglo o -1 si no existe.

Quiero que el código tenga comentarios explicando los dos casos base (éxito y fin de arreglo), la ausencia de bucles y el avance por índice.

También quiero probarlo con estos casos:

- Caso 1: arreglo = [10, 20, 30, 40], elemento a buscar = 30, índice inicial = 0 (debe devolver índice 2)
- Caso 2: arreglo = [10, 20, 30, 40], elemento a buscar = 10, índice inicial = 0 (debe devolver índice 0)
- Caso 3: arreglo = [10, 20, 30, 40], elemento a buscar = 99, índice inicial = 0 (debe devolver -1)